31

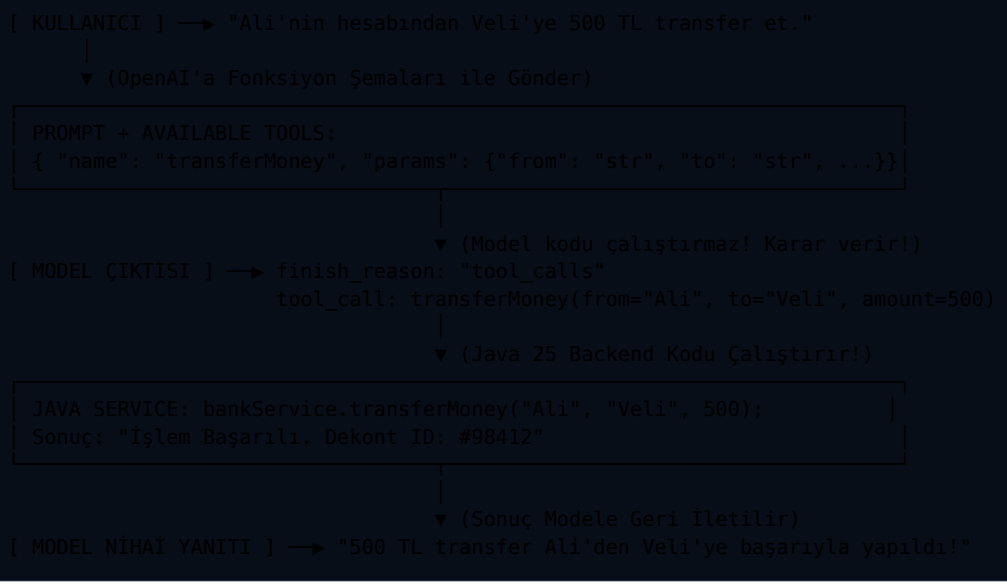
BÖLÜM: FUNCTION CALLING, TOOL USE VE API ENTEGRASYONU | MODULE: FUNCTION CALLING, TOOL INTEGRATION & AGENT TOOLING PROTOCOLS

Function Calling Nedir? LLM'lere Dış Dünyanın Anahtarlarını Vermek

What is Function Calling? Unlocking the External World for Foundation Models

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir yapay zeka kapalı bir cam fanusun içindedir; tek bildiği ekrandaki yazılardır. Dışarıdaki banka hesabınızı göremez, e-posta atamaz veya ışıkları yakamaz. Function Calling, bu fanusun kenarına bir kumanda panosu koymaktır! Siz modele 'Işıkları yak' dediğinizde, model kendi elleriyle ışığı yakmaz (çünkü elleri yoktur). Kumandadaki 'Işığı Yak' düğmesine basması gerektiğini anlar ve size 'Şu düğmeye bas' der (Tool Call). Sizin Java programınız o düğmeye basar ve sonucu modele fısıldar; model de size 'Işıklar yandı efendim' diye kibarca yanıt verir!

🇬🇧 ENGLISH (ELI5)

An LLM lives in an isolated soundproof glass room; it only processes text tokens. It has no physical hands to query a live MySQL database, execute payments, or trigger a webhook. Function Calling equips the model with a control panel! When you ask: 'Transfer $500', the model doesn't execute code directly. It reasons: 'I should invoke the transferMoney() tool with arguments A, B, and C.' Your secure Java backend executes the actual database transaction and hands the confirmation back to the model for natural language synthesis!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Function Calling Protokolü ve Döngüsü: 1) Tool Definition: API çağrısında modele JSON Schema formatında mevcut metodların adı, açıklaması ve parametre tipleri iletilir. 2) Tool Selection: Model ya düz metin üretir ya da `finish_reason: tool_calls` döner. 3) Execution: Uygulama (Java backend) gelen parametreleri doğrular ve gerçek servisi çalıştırır. Model KESİNLİKLE doğrudan sisteminizde kod koşturamaz! 4) Loop Closure: Fonksiyonun dönüş değeri `role: tool` mesajı olarak OpenAI'a geri gönderilir ve model son kullanıcıya nihai yanıtı üretir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Function Calling Execution Lifecycle: 1) Tool Manifest: Injected into the API payload as JSON Schemas detailing function signatures, parameter types, and docstrings. 2) Model Arbitration: The transformer decides whether to emit tokens or emit a `tool_calls` payload specifying target function names and JSON-encoded arguments. 3) Sandbox Execution: Your backend deserializes arguments, authorizes permissions, and executes the physical method (e.g. SQL, REST, Kafka). The model never directly touches your runtime. 4) Synthesis Round-Trip: Execution results are appended back into context with `role: tool` for conversational synthesis.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: Function Calling İstek ve Yanıt Protokolü Modeli
public record ToolCallRequest(String toolName, String jsonArguments) {}
public record ToolExecutionResult(String toolCallId, boolean isSuccess, String outputJson) {}

void main() {
    // Modelin ürettiği tool çağrısını simüle etme
    var simulatedModelCall = new ToolCallRequest("getWeather", "{\"city\": \"Ankara\"}");

    IO.println("Model Tool Çağrısı Yaptı: " + simulatedModelCall.toolName());
    IO.println("Parametreler: " + simulatedModelCall.jsonArguments());

    // Backend servisi çalıştırır
    var executionResult = new ToolExecutionResult("call_123", true, "{\"temp\": 18.5, \"condition\":
\"Sunny\"}");
    IO.println("Backend Çalıştırdı: " + executionResult.outputJson());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Güvenlik Açığı (Blind Execution): Modelden gelen parametreleri hiçbir doğrulama (Validation) yapmadan doğrudan SQL sorgularına veya para transfer metodlarına sokmayın! Model halüsinasyon görüp negatif bakiye (`amount: -500`) veya SQL enjeksiyonu gönderebilir. Girdiler daima Java katmanında valide edilmelidir.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Zero-trust validation rule: Never execute tool arguments emitted by LLMs blindly. Foundation models can be jailbroken or hallucinate invalid parameters (e.g. negative transfer amounts, SQL injection payloads). All tool arguments must pass strict schema and domain validation in Java.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• LLM'ler kodu kendileri çalıştırmaz; sadece hangi fonksiyonun çağrılması gerektiğini söyler.

• Gerçek iş mantığı daima güvenli Java backend katmanında yürütülür.

• Tool dönüş değerleri modele geri iletilerek döngü kapatılır.

🎯 Key Takeaways for the Interview

• LLMs do not execute code; they determine which tool to invoke with structured arguments.

• Physical execution occurs securely inside your Java application boundary.

• Tool outputs are fed back to the model to complete the multi-turn execution loop.